

fsb: Design Specification

September 21, 2026
Version 0.1 (draft)

Abstract

This document describes how **fsb** is built and why it was built that way: its structure, the mechanism behind each guarantee in the functional specification, the alternatives that were considered and rejected, how the design is tested, and what it does not do. It is written for people who maintain the program, review it, or test it adversarially. It changes when the construction changes; the functional specification does not. Requirement identifiers such as RUL-6 refer to the functional specification.

Contents

1	Principles	2
1.1	Technology	3
2	Architecture	3
2.1	Components	3
2.2	Structure	4
2.3	Life of a request	4
2.4	Startup	4
3	The Guard	4
3.1	Opening a path	4
3.2	Aliases	5
3.3	Roots	5
3.4	Permission errors inside denied places	6
3.5	Listing	6
3.6	Search	6
3.7	Cloud-only files	6
3.8	Other files and endpoints	6
3.9	Hard links	7
4	The Rule Matcher	7
4.1	Matching	7
4.2	Compiling a rule	7
4.3	Name equality	8
4.4	Speed	8
4.5	Loading a rule file	8
4.6	The built-in rules	8

5	The HTTP Layer	9
5.1	Middleware	9
5.2	The session and the prefix	9
5.3	Headers and framing	9
5.4	Streaming	9
5.5	Errors	10
5.6	Timeouts and shutdown	10
6	Content-Typed Endpoints	10
6.1	Images	10
6.2	PDFs	10
6.3	Archives	10
6.4	Rendered Markdown	11
7	The Front End	11
7.1	Structure	11
7.2	Decisions	11
7.3	The preview pane as a state machine	12
7.4	Hover peek	12
8	Verification	12
8.1	Layers	13
8.2	Continuous integration	13
8.3	What testing does not establish	13
9	Decisions and Alternatives	14
10	Limitations, Risks and History	15
10.1	Risks	15
10.2	Known limitations	15
10.3	History of defects found by the design’s own checks	16
10.4	Milestones	17

1 Principles

The security of `fsb` rests on a few principles that the rest of the design applies. Each was, at some point, the lesson of a defect (Section 10).

One chokepoint. Every read of the user’s file system goes through one package, the *guard*. No package that serves HTTP may touch the file system, and a test fails the build if one imports `os`, `io/fs`, `syscall` or `path/filepath`. A property of the guard therefore holds for every endpoint at once.

Fail closed. When a decision cannot be made, the answer is refusal: a malformed rule file stops the program from starting; a file with several names is refused until its names are known; a path that cannot be resolved is not served.

Over-match to deny, decide by identity to allow. Comparing names can be wrong in either direction. A deny check may safely match too much (it only blocks more); an allow check

must not, and so decides by the identity of the file or folder (device and inode), never by the spelling of its name.

Name equality belongs to the volume. Whether two names are the same file is a fact about the file system, not about the program. The program compares names the way the volume does, and tests that claim against the operating system rather than against its own model.

Decide from bytes, not names. What a file *is* (an image, a PDF, an archive) is decided from its contents. A name is at most a hint.

Read nothing that is not needed. Refusal is decided before any byte is read; a cloud-only file is never read implicitly, because reading it downloads it.

Each layer assumes the one before it failed. Rendered Markdown, for example, is escaped, then sanitized, then shown in a frame that cannot run anything of its own or reach the network.

Verify against the environment. The tests that matter most let the operating system decide the facts (Section 8).

1.1 Technology

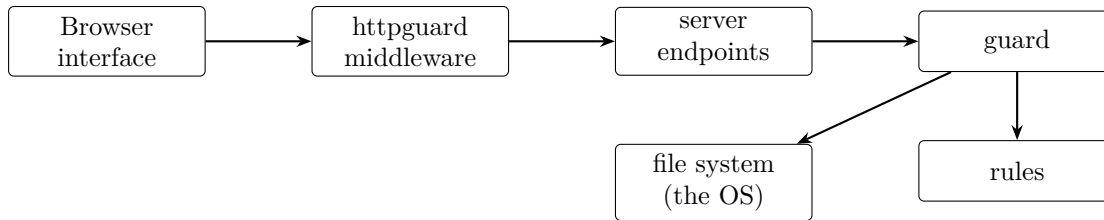
The back end is Go, using the standard library's HTTP server, with one extra dependency for Unicode (golang.org/x/text, for normalization and case folding) and one for extended attributes (golang.org/x/sys). The front end is Svelte 5 with TypeScript, built with Vite, and embedded in the binary with `go:embed`; the compiled output is committed so that building the binary needs no Node. The front end has three runtime dependencies that ship in the output: `marked` (Markdown), `dompurify` and `highlight.js`; Svelte is compiled away. The result is a single binary of about 11 MB.

2 Architecture

2.1 Components

Component	Responsibility
<code>cmd/fsb</code>	Command line, startup, the launch URL, opening the browser, shutdown.
<code>internal/rules</code>	The rule dialect: parsing, matching, name folding, the built-in rules. Knows nothing of files.
<code>internal/guard</code>	The chokepoint: opening, listing, searching, reading heads, metadata, sniffed endpoints, archive listing, the hard-link index. The only code that touches the file system on behalf of a request.
<code>internal/httpguard</code>	The middleware that makes a local web server safe: host and origin checks, the session, the prefix, security headers. Knows nothing of files.
<code>internal/server</code>	The endpoints. Turns guard results into HTTP, and guard errors into status codes.
<code>web</code>	The embedded front end, and the fixed page that renders Markdown.
<code>e2e</code>	Browser tests, and their drivers for Chrome and Safari.
<code>specs</code>	These documents.

2.2 Structure



Dependencies point one way: **server** uses **guard** and **httpguard**; **guard** uses **rules**; **rules** and **httpguard** use nothing of the program's own. **cmd/fsb** assembles them.

2.3 Life of a request

1. The HTTP server accepts the connection on the loopback address.
2. The access log records the method, path and status, never the query (which is where the launch token travels).
3. The middleware sets the security headers, then checks, in order: the **Host** header; the method; the **Origin**, **Referer** and **Sec-Fetch-Site** headers; that the path is under the random prefix (which it strips); the launch-token exchange; and the session cookie. A failure of any check is the same 403 (SEC-2 to SEC-8).
4. The endpoint reads its parameters and calls one guard operation.
5. The guard canonicalizes the path, checks it against the deny rules before and after opening, opens the file, verifies that what was opened is what was checked, and decides the file's kind from its bytes where needed.
6. The endpoint turns the result into a response, and every guard error into a status code in one function (**fail**), so that denied, missing and outside-the-roots paths cannot be told apart by accident.

2.4 Startup

cmd/fsb resolves the home folder to its real path, loads the two rule files (or their built-in defaults), builds the guard from the roots and rules, names the credential locations the guard must also protect, starts the hard-link index building in the background, binds the loopback port, prints the launch URL, and only then serves. A failure at any step before serving stops the program with a message.

3 The Guard

3.1 Opening a path

Every operation begins by opening the path, in this order. The order is the design.

1. **Canonicalize.** The path must be non-empty, contain no NUL, and be absolute. It is cleaned lexically (**.**, **..** and repeated slashes removed) and reduced to one spelling of each alias

(Section 3.2). URL-decoding has already happened, exactly once, in the HTTP layer; the guard never decodes, so a literal `%2e%2e` is just a name.

2. **Lexical deny check.** The cleaned path is matched against the deny rules *before the file system is touched*. The type of the final component is not yet known, so it is treated as a file here; a rule that applies only to folders is decided in step 7. (Assuming a folder would refuse a plain file that shares its name with a `dir/` rule although listings show it.)
3. **Open without blocking.** The path is opened read-only and non-blocking, so that a pipe or a device cannot hang the server. A permission error is examined (Section 3.4) before it is reported.
4. **Reject special files.** On the descriptor, anything but a regular file or a folder is refused. Only then is blocking restored.
5. **Resolve and confirm.** The real path is resolved (symbolic links followed), and the guard confirms that the file at that real path is the *same file* as the open descriptor (device and inode). If not, the path changed between the two steps and the request is refused as a race.
6. **Roots and deny on the real location.** The real path must lie inside a root (by identity, Section 3.3), and must not match a deny rule.
7. **Precise lexical check.** With the type now known, the requested path is matched again. A path is thus refused if *either* its spelling or its real location is denied.
8. **Other names.** A regular file with several names is checked against the hard-link index (Section 3.9).

Everything that later reads bytes does so through the descriptor this returns, so the decision and the read concern the same file. Extended attributes, for example, are read from that descriptor.

3.2 Aliases

macOS keeps its user data on a separate volume whose folders appear at ordinary paths through *firmlinks*: `/Users` and `/System/Volumes/Data/Users` are one directory. Firmlinks are not symbolic links, so resolving links does not reduce them, and a rule for `/Users/me/.ssh` would not match the longer spelling. Every path is therefore reduced to its ordinary form before any comparison with a root or a rule.

The reduction compares path components one at a time, with the same name equality as the rules (Section 4.3), and rebuilds the path from the firmlink's own spelling. An earlier version sliced the path to the byte length of the prefix, which cut a folded spelling (*/Slong-system*) in the middle of a character and left it unreduced. The firmlink table is read from `/usr/share/firmlinks`, with a built-in copy as fallback.

3.3 Roots

Each root is resolved to its real path at startup, and its identity (the file information of the folder) is kept. Whether a path lies inside a root is then decided by taking the leading components of the path that correspond to the root and comparing the *identity* of that folder with the root's. Comparing folded text would be right on a case-insensitive volume but would admit a different

sibling on a case-sensitive one (`Public` for a root of `public`). This is an allow decision, so it must not over-match.

3.4 Permission errors inside denied places

An unreadable file inside a denied folder must look exactly like a missing one, or its existence leaks. When opening fails with a permission error, the guard resolves the longest prefix of the path that can be resolved (an unsearchable folder stops the resolution of what is inside it, not of the folder itself), rebuilds the full real path from it, and applies the deny rules to that. Only if nothing denies it is the permission error reported.

3.5 Listing

A listing opens the folder as above and receives its real location. Each child is then judged by *that* location and not by the symbolic link or alias the folder was reached through, in one function used by both listing and search (`entry`), so that search cannot show what listing would not. For a child:

- a symbolic link is resolved; a broken one is marked; one whose target is denied or outside the roots is omitted; the target's size, date and type are shown;
- the child's path is matched against the deny rules and, separately, the hide rules, each ignoring the ancestors above the folder itself (so entering a hidden folder on purpose shows its contents);
- a regular file with several names is checked against the hard-link index.

Entries are handed out in batches as the directory is read, which is what lets the interface paint before a huge folder has been read.

3.6 Search

Search is a breadth-first walk, so shallow matches come first. Every folder is opened through the same operation, every child goes through `entry`, denied and hidden folders are never entered, and symbolic links to folders are never followed (which also rules out cycles). It stops at a result limit or an entry limit and reports that a limit stopped it. Names are compared after the folding of Section 4.3, or, for *Match case*, after normalization only.

3.7 Cloud-only files

On macOS a file that exists only in the cloud carries a flag in its file information. The guard checks it before any read and reports the file as not downloaded; the head, the metadata, the previews and the archive listing all refuse to read it. Reading such a file would make the system download it, which `fsb` must never do implicitly. An explicit download by the user is the one path that reads it.

3.8 Other files and endpoints

Head reads at most a stated number of bytes from the descriptor, and returns text only if it is valid UTF-8 without control characters; binary contents never leave the guard.

Metadata describes the entry, and reads extended attributes through the descriptor, capped in number and size.

Sniffed endpoints (image, PDF, archive) share one routine that opens the file, reads its first bytes, decides the kind from them, and rewinds; they differ only in the test applied (Section 6).

3.9 Hard links

Rules name paths, and a hard link is another name for the same file, so a hard link to a denied file is invisible to them. The guard therefore judges a regular file with more than one name by its identity: if any of its names is denied, all of them are.

The index. Finding the other names of a file means visiting every file under the roots, because the operating system offers no way to ask. A walk therefore builds an index from identity to names, containing only regular files with more than one name (a few dozen on a typical home folder), and the guard evaluates a file's names against the deny rules. The walk also covers the *protected locations* named by the program at startup (the credential folders of every home), so that a link from a served folder to a credential file outside the roots is found.

The walk is in the background. On a home folder of 700,000 files it takes about 18 seconds, which must not be spent inside a request. It is started at startup. While it has not finished, and for a file with several names that it has not seen (a link made since), the file is *refused*: guessing "allowed" would defeat the purpose. A later walk that began after the file was first noticed and still does not find it shows that the walker cannot see it, and the file is then allowed, so that nothing is refused forever. Walks are spaced by at least twice the time the last one took, and bounded by an entry count and a time.

What is not covered. A denied name that lies outside every root and every protected location, and links made after a walk was stopped by its limit, are not found.

4 The Rule Matcher

4.1 Matching

A rule set is an ordered list of compiled rules. For a path, the matcher evaluates each prefix of the path in turn (the folder itself, its parent, and so on down to the path), and the set *excludes* the path if any prefix is excluded: an excluded folder excludes everything in it, and no later rule (not even a negation) can bring it back, as in git. Within one prefix the last matching rule wins, which is what makes negation in the ignore file work. The deny file has no negation, so it is a plain union of rules.

Matching a path *below* a folder ignores the prefixes at or above that folder. It is used for the hide rules while listing, so that entering a hidden folder on purpose shows what is inside it.

4.2 Compiling a rule

A rule is translated to a regular expression, case-insensitive and with the dot matching a newline (so that a name containing a newline cannot defeat ******), ending in **\s*\$**, so that a name with trailing white space still matches. The translation follows the gitignore glob: ***** and **?** do not cross a separator; ****** matches folders; a class **[...]** never matches a separator (a negated class excludes it explicitly, and a **/** inside a class is an error); a trailing **/** makes the rule apply to folders only. A pattern is anchored to the home folder if it begins with **~/**, is absolute if it begins with **/**, is

relative to the home folder if it contains a `/` elsewhere, and matches at any depth otherwise or after a leading `**/`.

4.3 Name equality

Both the pattern and the path are folded before they are compared (**Fold**): Unicode normalization (NFC), then *full* case folding, then NFC again. The rules must match names as the volume compares them, and on macOS the case-insensitive volume uses full folding, under which the sharp `s` equals `ss`, the long `s` equals `s`, the Kelvin sign equals `k`, and the ligatures `ff`, `fi`, `fl`, `ffi`, `ffl` and `st` equal their spelled-out forms. Simple case folding, which was used at first, treated these as different names, so `~/fsh` opened `~/ssh` while the rule for `.ssh` did not match. The set of aliases was *measured* on the volume, by creating every one- and two-letter name and probing every character in the Unicode range, and is a test. The client-side filter uses the equivalent lower-, upper-, lower-casing, and the tests check that the two agree on the measured aliases.

Names are compared this way so that a deny rule over-matches, which is safe. Where the result is an *allow* (whether a path is inside a root), identity is used instead (Section 3.3).

4.4 Speed

Listing a folder evaluates the rules once per entry. A rule set of about thirty rules took about 29 μ s per entry, which would add three seconds to a folder of 100,000 entries, so two optimizations were made, both of which must give the same answer as the plain loop:

- A rule whose pattern contains no slash can match only the last component of a path, so it is tested against that component alone (short and anchored) instead of against the whole path at every level.
- For a set without negation, all rules of each kind are combined into one expression, asked first; only when it says a match is possible are the rules consulted one by one, to say *which* rule matched (for `-debug`).

The cost is about 9 μ s per entry, and a property test checks that the quick path never changes an answer.

4.5 Loading a rule file

A rule file is written by a person in an editor, and a deny rule that loads but never matches is the worst outcome. The parser therefore either normalizes or refuses. It strips a leading byte order mark (which otherwise broke the first rule) and accepts any line ending (a bare carriage return joined lines). It refuses, naming the line: an inline comment, `~name`, a quoted pattern, an invisible character, a lookalike of the slash, negation in the deny file, and the malformed patterns of the functional specification (RUL-12). Whatever loads, works as written.

4.6 The built-in rules

The core deny rules are compiled into the binary, so they apply even without a deny file. The credential rules are not anchored to the current home folder, because the same folders exist in backups, clones, other accounts' homes and under a wrong **HOME**. If a deny file exists it is authoritative, and **CoreMissing** compares the core rules with the file's rules *as names* to decide which are not in

effect. The list of credential locations for the hard-link index (Section 3.9) mirrors the credential entries of the core rules.

5 The HTTP Layer

5.1 Middleware

The middleware is a single function that either refuses a request or passes it on with the prefix removed. Its checks, and the order of them, are in the life of a request (Section 3); this section records the reasons.

Host. A page on another name that resolves to the loopback address still sends its own name in the `Host` header, so an exact allowlist of the two loopback names with the port defeats DNS rebinding. The port is fixed at start.

Origin, Referer, Fetch metadata. A browser serializes `Origin` as exactly scheme, host and port; anything else (a path, credentials, an opaque `null`) is refused rather than interpreted. `Sec-Fetch-Site` must be `same-origin` or `none` (or absent, for clients that do not send it). The value `same-site` is refused because a page on another port of the same host is same-site to the browser, so `SameSite=Strict` does not stop it.

Read-only. Only GET and HEAD reach a handler; the endpoint table contains no other method.

5.2 The session and the prefix

The launch token and the session value are 256-bit random values, compared in constant time. The token is exchanged once, atomically, for the session cookie; the response redirects to the prefixed root so the token leaves the address bar and the history.

A cookie is scoped by host and by path, never by port. A plain `Path=/` cookie would be sent to every other web server on 127.0.0.1 that the same browser visits, and a server that received it could replay it. The design therefore serves everything under a random 128-bit path prefix chosen at start, sets the cookie's path to it, and refuses (403) any URL not under it. The browser then sends the cookie only for URLs under the prefix, which no other server has. The prefix is a scope, not a secret: the session cookie is still required. The front end forms every URL relative to its own location so that this needs no configuration.

5.3 Headers and framing

Every response carries the headers of SEC-9. Two endpoints relax framing, deliberately and narrowly: the fixed Markdown page and the PDF may be framed by the interface's own pages (`X-Frame-Options: SAMEORIGIN, frame-ancestors 'self'`), and the interface itself may not.

5.4 Streaming

Listings and searches are newline-delimited JSON, flushed as batches are ready, because a folder can hold hundreds of thousands of entries and the first screen must not wait for the last. A listing that breaks after it has begun ends with an error line rather than a status code, since the status was already sent. Pagination was rejected: the client sorts, and a sort needs all entries, so streaming keeps the whole listing available and still paints early.

5.5 Errors

One function maps every guard error to a status. Denied, outside-the-roots, nonexistent, malformed and not-regular all map to the same 404 with the same body; only `-debug` adds the rule name. Permission errors on paths that passed every rule map to 403; cloud-only files to 409; wrong kinds to 415; anything unexpected to 500 with the detail in the log only.

5.6 Timeouts and shutdown

The server sets a header-read timeout against slow clients and has no write timeout, because streams may legitimately last. On interrupt or termination it stops accepting requests and gives requests in progress three seconds.

6 Content-Typed Endpoints

Three endpoints look inside a file: inline images, inline PDFs, and archive listings. Each is defined by a test on the file's *leading bytes*, never its name, and each opens the file through the guard, so every property of Section 3 applies to it unchanged.

6.1 Images

An image is served inline only if its first bytes are a PNG, JPEG, GIF or WebP signature. The response is sent with a policy that sandboxes it and allows no resources, so even a crafted file has nothing to reach. SVG is never accepted, because it can carry script, and neither is HTML. A file named `.png` that is not one is refused (415) and the interface falls back to the ordinary file preview.

6.2 PDFs

A PDF is served inline only if it begins with `%PDF-`. It cannot be sent with the `sandbox` policy that images get, because browsers then refuse to display it. The protection is instead the byte check, framing headers that allow only the interface's own pages to embed it, a policy that forbids script in the response, and the browser's own PDF viewer, which runs in an isolated component of its own. The interface asks first, with a one-byte range request, whether the file really is a PDF, and only then points a frame at it.

6.3 Archives

An archive listing returns names and sizes only, never member data, and nothing is extracted. The format is decided from bytes: the zip signature, the `ustar` marker at offset 257 of a tar, or a gzip stream that contains a tar. The limits of LIM-1 bound a hostile file:

- A zip's directory is held in memory by the standard library, so the guard first reads the end-of-central-directory record and refuses a zip that claims more entries than it will index (and, when the count is stored elsewhere in a very large zip, refuses by size).
- A tar is streamed, stopping at an entry limit; a gzip-compressed tar is read through a reader capped at a number of decompressed bytes, so a bomb costs a bounded amount of time and memory, and the listing says it was cut short.

- Member names are untrusted: control characters and invalid UTF-8 are replaced, and the interface additionally replaces bidirectional characters, and names are only ever displayed, never used as paths.

6.4 Rendered Markdown

Rendering a Markdown file must not run code, load anything remote, or reach the interface. It is done in three layers, each assuming the one before failed.

1. **Escaping.** The Markdown is converted by a library configured so that raw HTML in the source becomes text; links carry no address at all (an attribute says what they mean: a file on the disk, or a web address); images carry no source (a marker names a local file); a remote or unsupported image becomes a text placeholder. A link is resolved against the Markdown file's folder to a normalized absolute path, or dropped to plain text if it is not a relative path or an `http` or `https` address.
2. **Sanitizing.** The result goes through DOMPurify with a fixed list of allowed tags and attributes.
3. **A sandboxed frame.** The sanitized markup is sent by `postMessage` to a fixed page, framed with `sandbox="allow-scripts"` (so it has an opaque origin and no access to the session or the interface). The page's own policy is `default-src 'none'` with images limited to `data:`, and admits its one script and one style *by hash*, computed from the page at start so that a change to the page cannot silently invalidate the policy. The page accepts messages only from its parent.

Links are not followed inside the frame. A click sends a message to the interface, which validates it and either changes the location to the target file's folder with the file selected, or opens a web address in a new tab without a referrer. Images inside the roots are fetched by the interface, through the guarded image endpoint, and passed in as `data:` URLs, because the cookie's scope and `SameSite` would otherwise stop the opaque-origin frame from fetching them. Opening a Markdown file therefore makes no request to anything outside the machine.

7 The Front End

7.1 Structure

The interface is a single page built with Svelte 5 (runes), TypeScript and Vite. Its logic is kept in small *pure* modules, which import nothing from the browser, so that they run under Node's own test runner: ordering (`sort`), filtering (`filter`), column layout (`columns`), preferences (`prefs`), preview classification and text helpers (`preview`), the path and location functions (`format`, `goto`), the key mapping (`keys`), the Markdown conversion (`markdown`), and where the page was loaded from (`base`). The components hold only what needs the DOM.

7.2 Decisions

Virtualized list. Only the rows on screen are drawn, from a scroll position and a fixed row height; a folder of 100,000 entries draws about 30 rows.

Location as state. The current folder, and the entry to select, are in the URL fragment (`#/path?select=name`), which is never sent to the server, so folders are bookmarkable and Back works without any routing library. Parsing is strict: a bad escape is not silently replaced.

Requests relative to the page. Every request is formed from the page's own directory, so the random prefix (which scopes the cookie) needs no configuration.

Sorting in the browser. The server streams entries in the order the file system gave them; the browser sorts, with a comparison that is a total order and does not depend on the input order (a test). A time that cannot be parsed sorts as the oldest, so the order stays total.

Names are displayed, not trusted. Characters that would disguise a name are replaced by visible markers for display only. Names are always inserted as text, never as markup.

Preferences are optional. They are kept in local storage with every read guarded, and every value falls back on its own.

No dynamic code. There is no use of `eval`, of `innerHTML`, or of any other markup sink except one: the output of the syntax highlighter, which escapes its input.

7.3 The preview pane as a state machine

For a selected entry the pane is in one of a few states (idle, loading, folder, image, PDF, archive, text, binary, empty, cloud-only, error). A file name gives a *hint*, the server decides from the bytes, and a hint the server rejects falls back to the ordinary head request, which never falls back again, so every run ends in a terminal state after at most two requests. Every request started for a selection, including a fall back, shares one cancellation handle and is cancelled when the selection changes, so a late answer can never be shown for the wrong file. The laws are in the algebraic specification.

7.4 Hover peek

A short delay before the request, a cache, a limit on simultaneous requests, and cancellation on pointer movement, scroll or key press. It requests the same head endpoint as the preview, so it is subject to the same rules, and is off if the user turns it off.

8 Verification

The strategy follows the principles: the parts that decide what is served get the heaviest testing, and the tests that matter most let the operating system decide the facts.

8.1 Layers

Layer	What it establishes
Unit and integration (Go)	Behavior of the matcher, the guard, the middleware and every endpoint, including deliberately hostile paths. A test fails the build if an HTTP-facing package imports <code>os</code> , <code>io/fs</code> , <code>syscall</code> or <code>path/filepath</code> .
Property tests	The laws of the algebraic specification, with fixed seeds: monotonicity and order independence of deny, ancestor closure, fold invariance, listed implies openable, found implies listed, and the equivalence of the quick matching path with the plain one.
Fuzzing	The guard (with an oracle that does not trust <code>fsb</code> 's rules: whatever it returns must not be the same inode as a known secret or contain a secret marker), the matcher, the middleware, and the whole server stack.
Differential test against the OS	For each secret, thousands of generated spellings (case, folding aliases, decomposed accents, separators, dot segments, firmlink and symbolic-link aliases, named forks) are filtered by the operating system to those that really reach the secret, and every one must be refused. The reverse direction requires every such spelling of an ordinary file to be served. Reverting the folding fix makes it fail about 1,500 times.
Real-volume tests	Root containment on a real case-sensitive volume, run when a location for one is given.
Front-end tests (Node)	The pure modules, including the laws for ordering, filtering, location round trips, link resolution and name display.
Browser tests	A dependency-free suite written once against a driver interface, run with real keyboard and mouse events in Chrome (DevTools protocol) and Safari (WebDriver). It includes attacks from a second origin and from a page on another port, a DNS-rebinding hostname, the cookie's scope, and a 100,000-file folder.
Mutation checks	Removing a rule or reverting a fix must make a test fail; this was done for the folding fix, the cookie path, and others.
Algebra	The laws, and the defects found by stating them (Section 10).

8.2 Continuous integration

On every change: formatting, vetting, the Go tests under the race detector, the front-end tests, type checking, a build whose output must equal the committed one, an audit of the packages that ship, the Chrome browser tests, and timed fuzzing of the four fuzz targets. A scan for known vulnerabilities in the Go modules runs as well. (The workflow has not yet run on a hosted service, because the repository has no remote yet.)

8.3 What testing does not establish

None of this is a guarantee. It is a record of what was checked. A fuzzer reaches only what its fixture contains, and an early version missed a bug for that reason; the differential test exists because reasoning about names had been wrong. An independent review, and a professional penetration test before wide release, remain planned.

9 Decisions and Alternatives

Each row is a decision that shaped the design, with what else was possible and why it was not chosen.

Decision	Rejected	Why
Write from scratch in Go with an embedded Svelte front end	Fork an existing multi-user file manager	The existing one is built around a jailed root, configuration in its own UI, and create, rename and delete. The security model here (whole home, read only, two-tier rules) is different in kind, and a fork would carry code paths that must not exist.
Two tiers: hide and deny	One list	Hiding clutter is cosmetic and must not be a security control; blocking must be absolute, not overridable, and load only from the global file so that a folder cannot relax it. Deny wins over ignore.
Deny loads only from the global file	Per-folder deny files	A per-folder file could be planted in the tree being browsed to weaken protection.
Core rules built in and active by default	Ship them commented out, with a warning	A tool that exposes <code>~/ssh</code> until configured is unsafe by default. The warning now covers only the case where the user weakened them.
Bind loopback only, not configurable	A flag for other interfaces	There is no safe way to serve the whole home folder to a network, so the option is not offered.
Single-use launch token, then a cookie	A permanent token in the URL	A token left in history or a shell must be useless.
Path prefix scoping the cookie	A plain cookie; a custom header on every request	Browsers cannot scope cookies by port. A header cannot accompany images, frames, PDFs and downloads. A random path scopes the cookie exactly.
Identity for hard links, from a background index	A hash of contents; a synchronous walk	A hash flags harmless copies, reads every candidate, and answers a different question. A synchronous walk did not finish in four seconds on a real home folder and would stall a request. Refusing until known keeps it safe meanwhile.
Roots decided by identity	Comparing folded text	Right on a case-insensitive volume, wrong on a case-sensitive one.
Full case folding, from <code>x/text</code> , applied to rules	<code>ToLower</code>	The volume uses full folding; simple folding made some spellings of a denied name invisible to the rules.
Refuse a file whose kind the bytes do not confirm	Trust extensions	An extension is chosen by whoever named the file.
Markdown in a sandboxed frame, sanitized, with a hash-pinned policy	Render in the page with a sanitizer only	One layer failing must not compromise the session; the frame has no origin, no network and one fixed script.
PDF in the browser's viewer in a frame with narrow framing headers	Bundle a PDF library; sandbox the frame	A library is large and a second parser to defend; a sandboxed frame is refused by browsers.
Stream listings as newline-delimited JSON	Pagination	The client sorts, which needs all entries; streaming paints early and keeps them all.
Sort in the browser	Sort on the server	The sort key and options change often; the total-order comparison is a pure, testable function.

Decision	Rejected	Why
Pure modules tested with Node's runner	A browser test for every function	Fast, dependency-free, and the laws are stated on them.
Commit the compiled front end	Build it in the Go build	Building the binary must need no Node. CI checks that the committed output is current.
Deny checks over-match; allow checks decide by identity	One equality for both	The two directions fail differently: over-matching a deny only blocks more, over-matching an allow serves the wrong folder.
Refuse files with several names until the index is ready	Allow until known	The window in which a hard link to a secret is served must be zero, at the cost of a short unavailability.
Hand-written manual page in roff	Generate it from Markdown	No build dependency, and the format users expect.

10 Limitations, Risks and History

10.1 Risks

- R1: sensitive paths are exposed by default.** The root is the home folder, so without protection `~/.ssh` and similar would be reachable. Mitigated by the core deny rules, active by default and built in, and by a startup notice and a banner if the user weakens them. Residual: a secret with an unremarkable name outside the core list, the optional rules left off, and a user who disables the core rules and ignores the warning.
- R2: files with several names.** Path rules cannot see a second name for a file. Covered for denied names under the roots and for the credential locations; see the limits below.
- R3: untrusted content in a page that holds a session.** Markdown, PDF and archive names are rendered or shown; mitigated by the layers of Section 6.
- R4: permission prompts.** macOS asks permission the first time for Documents, Desktop, Downloads and external volumes; the documentation says so.

10.2 Known limitations

- A denied name outside every root and every protected credential location is not found as another name of a file; links created after a size-limited walk are not found.
- Response timing is not equalized: a denied path can be answered slightly faster than a missing one. The status and body are identical.
- Anything that runs as the user can read the same files. `fsb` is not a sandbox against local malware.
- The metadata endpoint reads a symbolic link's text from the requested path after the file was opened; an attacker who could rewrite that link during the call could learn the text of a denied path (not its contents). Requires a same-user attacker.
- Rule files are read at startup only.

- Not yet tried with real cloud-placeholder files or in Firefox; a real mouse drag of a column header in Safari is unverified.
- The browser test for Safari occasionally fails in a run of tests that need a focused window, and passes on rerun.

10.3 History of defects found by the design's own checks

Stating the laws of the algebraic specification, and then auditing every comparison of names and paths, found sixteen defects, all fixed and each covered by a test. In brief:

1. Simple case folding let `.Bh` open `.ssh` (a bypass of a deny rule).
2. A plain file named like a folder-only rule was listed but could not be opened.
3. A negated glob class could match across a path separator.
4. A row order that was not transitive when a time could not be parsed.
5. The filter and the search disagreed about the sharp `s`.
6. A stale fallback request could show the wrong file's preview.
7. Firmlink prefixes were matched by byte length (a bypass under a root of `/`).
8. Root containment compared spellings (wrong on a case-sensitive volume).
9. Breadcrumbs, selection and the core-rule warning compared names by spelling.
10. The credential rules were anchored to one home folder.
11. A hard link to a credential file was served.
12. The session cookie followed the host, not the port.
13. A hard link to any other denied file was served.
14. Rule files that loaded without error and did nothing.
15. Secrets under other names (`id_rsa`, certificate stores, password databases).
16. File names that disguised themselves with bidirectional characters.

The lesson common to several: name equality is a property of the volume, so deny checks may over-match but allow checks must decide by identity.

10.4 Milestones

Milestone	Scope	Status
M0 Skeleton	Repository, CI, guard, matcher, middle-ware	Done.
M1 Crawl	Browsing, listing, filtering, sorting, rules	Done; 100,000-file folder measured. Daily use by the author still to confirm.
M2 Walk	Preview, hover peek, search, attributes, columns, rendered Markdown, PDF, archives	Built and tested in Chrome and Safari; not yet on real cloud placeholders or in Firefox.
Hardening	The audits and defects of the preceding subsection	Done; a professional penetration test is planned near release.
M3 Public release	Documentation, releases, Homebrew formula	Not started; packaging is done last.
M4 Run	The stage-three requirements of the functional specification	Reassess after real use.